



Alexandria University
Faculty of Engineering — Computers & Systems Engineering
Designing Data Intensive Applications | CSE-4E3

Weather Stations Monitoring

Project Report

Abdelrahman Amr Ali	9010
Abdelrahman Ahmed Hafez	8901
Abdelrahman Hussein Attia	8976
Youssef Mohamed Omar	8821

1. System Architecture

Overview of the end-to-end pipeline: weather stations → Kafka → Central Station → BitCask + Parquet → Elasticsearch / Kibana.

2. Weather Station Mock

2.1 Message Schema

Each station emits one JSON status message per second with the following fields:

- station_id — unique station identifier (Long)
- s_no — auto-incremental sequence number (Long)
- battery_status — low / medium / high (30% / 40% / 30%)
- status_timestamp — Unix timestamp (Long)
- weather — humidity (%), temperature (°F), wind_speed (km/h)

2.2 Randomisation & Drop Logic

Messages are randomly dropped at a 10% rate before being sent to Kafka.

2.3 Key Implementation Details

Built in Java 11, using Jackson's ObjectMapper to structure the JSON payloads cleanly. To ensure a reliable 1-second emission cycle without overcomplicating the threading model, we used a standard while(true) loop paired with Thread.sleep(1000). The 10% message dropping and the 30/40/30 battery distribution were handled using standard Java random number generation right before the message is passed to the Kafka producer.

3. Kafka Integration

3.1 Producer Setup

The producer connects to our containerized Kafka broker at 127.0.0.1:9092. Since we are pushing plain JSON payloads, we went with standard StringSerializer for both the keys and the values to keep things lightweight. All raw mock data is published to a main topic named weather_statuses, which sets up a clear, single source of truth for the Streams processor to read from.

3.2 Rain Detection Processor

A Kafka Streams processor monitors humidity. When humidity > 70%, a rain-alert message is forwarded to a dedicated topic.

4. Central Station

4.1 BitCask Riak — Key-Value Store

Stores the latest status for each station_id. Implementation highlights:

- Append-only segment files with in-memory keydir
- Hint files for fast keydir reconstruction on restart
- Background compaction scheduled without blocking active readers

4.2 BitCask Client

A bash/Python CLI with three modes:

- `--view-all` → dumps all keys + values to a timestamped CSV
- `--view --key=K` → prints latest value for key K to stdout
- `--perf --clients=N` → spawns N concurrent threads, each running `--view-all`

4.3 Parquet Archiving

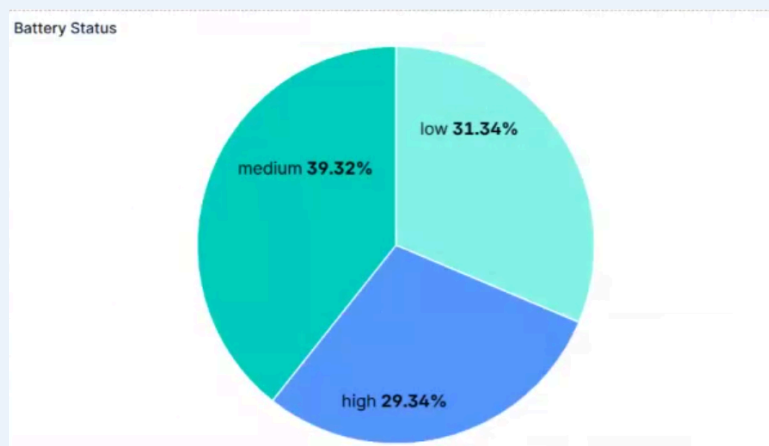
All received statuses are written to Parquet files partitioned by time and station_id. Records are batched (~10 K) before flushing to avoid frequent I/O.

5. Elasticsearch & Kibana Analysis

Parquet files are connected as a data source for Elasticsearch. Kibana dashboards confirm the simulated distributions.

5.1 Battery Status Distribution

Expected: 30% low — 40% medium — 30% high across all stations.



5.2 Dropped Messages per Station

Expected: ~10% message drop rate per station.

Percentage is calculated using this formula

$$(\max(s_no) - \text{count}(s_no)) / \max(s_no)$$

Dropped Message Rate		
	station_id	Dropped Percentage
1	1	8.82%
2	2	10.55%
3	3	7.39%
4	4	5.98%
5	5	14.29%
6	6	19.49%
7	7	13.56%

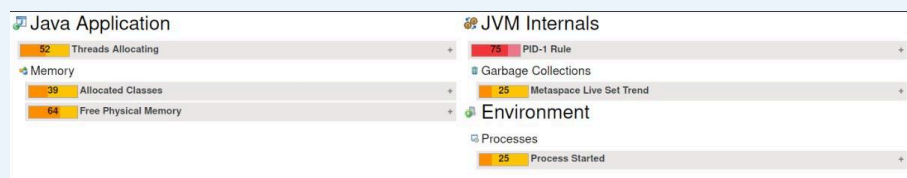
6. Kubernetes Deployment

All components are containerised and deployed via Kubernetes:

- 10 weather-station pods (weather-station image)
- 1 central-station pod (central-server image)
- Kafka + Zookeeper services (Bitnami image)
- Elasticsearch + Kibana service
- Shared PersistentVolume for Parquet files and BitCask LSM

7. JFR Profiling — Central Station

The Central Station was profiled with Java Flight Recorder for 1 minute under full load (7 stations).



centraljfr x

GC Summary

Young Collection Total Time		Old Collection Total Time	
GC Count	3	GC Count	1
Average GC Time	3.208 ms	Average GC Time	17.940 ms
Maximum GC Time	4.347 ms	Maximum GC Time	17.940 ms
Total GC Time	9.623 ms	Total GC Time	17.940 ms

All Collections Total Time		All Collections Pause Time	
GC Count	4	Average Pause	2.510 ms
Average GC Time	6.891 ms	Longest Pause	4.347 ms
Maximum GC Time	17.940 ms	Sum of Pauses	12.550 ms
Total GC Time	27.563 ms		

centraljfr x

Live Objects

Allocation 10⁴

5/23/2026 10:11:50 AM 10:12:00 AM 10:12:10 AM 10:12:20 AM 10:12:30 AM 10:12:40 AM

Live Object Sample	Count	Description
ConcurrentHashMap\$Node[]	1	
Class	1	
byte[]	1	
byte[]	1	
byte[]	1	
byte[]	1	
byte[]	1	
byte[]	1	
byte[]	1	
byte[]	1	
byte[]	1	

8. Bonus

8.1 Open-Meteo Integration

Used Java 11's native HttpClient to build a Channel Adapter that pings the Open-Meteo API every 5 seconds. The adapter grabs real-time weather data, translates it into our exact project JSON schema, and assigns it a dedicated identifier (Station 99). It then pushes this live data directly into the weather_statuses Kafka topic, seamlessly feeding real-world data alongside our simulated sensors.

8.2 Enterprise Integration Patterns

The following EIP patterns were applied:

- Polling Consumer: Elastic search ingestor polls over the shared folder to watch for parquet files
- Idempotent Receiver: Elastic search ingestor moves processed parquet files to separate folder

- Dead Letter Channel: Elastic search ingestor moves corrupted parquet files to separate folder